

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA (FIAP)

Tecnólogo em Análise e Desenvolvimento de Sistemas

André Rosa Colombo - RM563112 | 2TDSA

Gabriely Bonfim Silva - RM566242 | 2TDSA

Henrique Rodrigues Vespasiano - RM562917 | 2TDSA

Mirelly Sousa Alves - RM566299 | 2TDSA

Ruan Luca Feliciano - RM562218 | 2TDSA

Equipe: Olli Pet

Challenge Clyvo

Mastering Relational and Non-Relational Database

São Paulo

2026

Sumário

1. Modelo Descritivo.....	3
2. Modelo Lógico (DER).....	6
3. Modelo Físico (MER).....	8
4. Implementação SQL.....	9
4.1. Estrutura e carga de dados.....	9
4.2. Procedures.....	10
4.2.1. P_RELATORIO_JSON.....	10
4.2.2. P_RELATORIO_TRATAMENTOS.....	10
4.3. Functions.....	11
4.3.1. F_JSON_ATENDIMENTO.....	11
4.3.2. F_FIM_TRATAMENTO.....	12
4.4. Trigger.....	13
4.5. Consultas Complementares.....	14
5. Observações.....	17

1. Modelo Descritivo

O banco de dados do sistema Clyvo foi estruturado para armazenar informações relacionadas a responsáveis, animais, profissionais veterinários, consultas, prontuários, vacinas, aplicações de vacinas e tratamentos. Também foram criadas estruturas específicas para o registro de erros e para a auditoria das alterações realizadas nos prontuários.

Responsável

Armazena os dados dos responsáveis pelos pets.

- **id_resp:** identificador único do responsável.
- **nome:** nome completo do responsável.
- **data_nasc:** data de nascimento.
- **cpf:** documento do responsável, com restrição de unicidade.
- **email:** endereço de e-mail.

Médico Veterinário

Armazena os dados dos profissionais responsáveis pelos atendimentos.

- **id_med_vet:** identificador único do médico veterinário.
- **nome:** nome do profissional.
- **crm_v:** registro profissional, com restrição de unicidade.
- **especialidade:** área de atuação do profissional.
- **email:** endereço de e-mail.

Pet

Armazena os dados dos animais cadastrados no sistema.

- **id_pet:** identificador único do pet.
- **nome:** nome do animal.
- **descricao:** características ou observações sobre o animal.
- **raca:** raça do animal.
- **especie:** espécie do animal.
- **data_nasc:** data de nascimento.
- **id_resp:** referência ao responsável pelo pet.

Consulta

Registra os atendimentos agendados ou realizados para os pets.

- **id_consulta:** identificador único da consulta.
- **id_pet:** referência ao pet atendido.
- **id_med_vet:** referência ao médico veterinário responsável.
- **data_hora:** data e horário da consulta.
- **motivo:** motivo do atendimento.
- **status:** situação da consulta.

- **observacoes:** informações adicionais sobre o atendimento.

Prontuário

Armazena os registros dos procedimentos realizados durante os atendimentos.

- **id_prontuario:** identificador único do prontuário.
- **id_pet:** referência ao pet relacionado ao prontuário.
- **id_med_vet:** referência ao médico veterinário responsável.
- **id_consulta:** referência à consulta relacionada.
- **procedimento:** descrição do procedimento realizado.
- **data_procedimento:** data do procedimento.
- **local_atendimento:** local onde ocorreu o atendimento.
- **observacao:** informações adicionais sobre o procedimento.

Vacina

Armazena os protocolos de vacinação disponíveis.

- **id_vacina:** identificador único da vacina.
- **nome:** nome da vacina.
- **especie:** espécie para a qual a vacina é destinada.
- **doses_protocolo:** quantidade de doses previstas no protocolo.
- **intervalo_dias:** intervalo entre as doses.
- **meses_reforco:** período previsto para reforço.

Aplicação de Vacina

Registra as aplicações de vacinas realizadas nos pets.

- **id_aplicacao:** identificador único da aplicação.
- **id_pet:** referência ao pet vacinado.
- **id_vacina:** referência à vacina aplicada.
- **id_med_vet:** referência ao médico veterinário responsável.
- **numero_dose:** número da dose aplicada.
- **data_aplicacao:** data da aplicação.
- **proxima_dose:** data prevista para a próxima dose.
- **lote:** identificação do lote da vacina.

Tratamento

Registra os tratamentos prescritos para os pets.

- **id_tratamento:** identificador único do tratamento.
- **id_pet:** referência ao pet em tratamento.
- **id_med_vet:** referência ao médico veterinário responsável.
- **id_consulta:** referência à consulta relacionada.
- **medicamento:** medicamento ou recurso utilizado no tratamento.
- **dosagem:** dosagem ou forma de utilização.
- **intervalo_horas:** intervalo entre as administrações.

- **duracao_dias:** duração prevista do tratamento.
- **inicio_em:** data de início do tratamento.
- **status:** situação atual do tratamento.
- **observacoes:** informações adicionais.

Log de Erros

Armazena as informações dos erros tratados durante a execução das procedures.

- **id_log:** identificador único do registro.
- **nome_procedure:** nome da procedure em que ocorreu o erro.
- **usuario_db:** usuário do banco de dados responsável pela execução.
- **data_erro:** data e horário do registro.
- **codigo_erro:** código do erro retornado pelo banco.
- **msg_erro:** descrição do erro ocorrido.

Auditoria de Prontuário

Registra as operações realizadas na tabela de prontuários, permitindo identificar inclusões, alterações e exclusões.

- **id_auditoria:** identificador único do registro de auditoria.
- **usuario_db:** usuário responsável pela operação.
- **operacao:** tipo de operação realizada, sendo INSERT, UPDATE ou DELETE.
- **data_hora:** data e horário da operação.
- **campos anteriores:** valores existentes antes de uma alteração ou exclusão.
- **campos novos:** valores inseridos ou resultantes de uma alteração.

2. Modelo Lógico (DER)

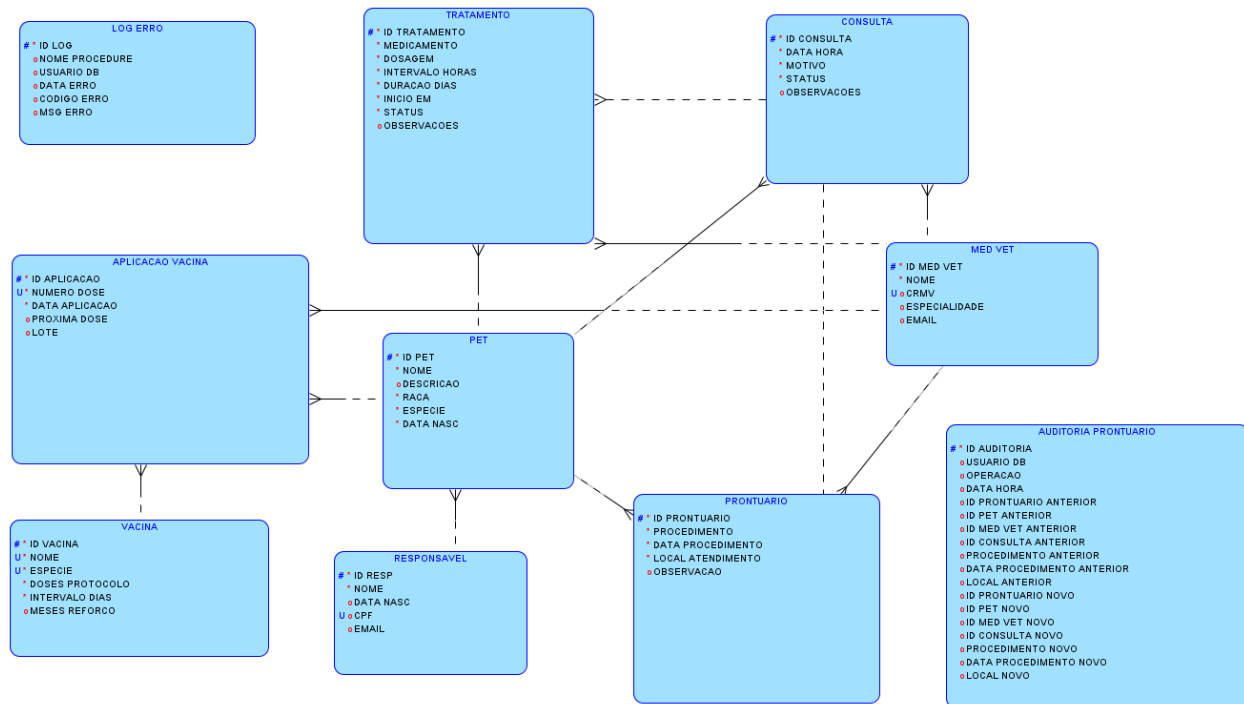
O Modelo Lógico representa a estrutura relacional do banco de dados e os relacionamentos existentes entre suas entidades. O diagrama foi desenvolvido utilizando a notação Barker, apresentando as entidades, seus atributos, chaves e cardinalidades.

O modelo possui como entidades principais **Responsável, Pet, Médico Veterinário, Consulta, Prontuário, Vacina, Aplicação de Vacina e Tratamento**, além das estruturas destinadas ao registro de erros e à auditoria.

Os principais relacionamentos são:

- Um **responsável** pode possuir um ou vários pets, enquanto cada pet está associado a um responsável.
- Um **pet** pode possuir várias consultas, enquanto cada consulta está relacionada a um pet.
- Um **médico veterinário** pode realizar várias consultas, enquanto cada consulta possui um médico veterinário responsável.
- Um **pet** pode possuir vários registros de prontuário.
- Um **médico veterinário** pode estar associado a vários registros de prontuário.
- Uma **consulta** pode estar relacionada a um registro de prontuário.
- Um **pet** pode possuir várias aplicações de vacinas.
- Uma **vacina** pode ser aplicada em vários pets.
- Um **médico veterinário** pode realizar várias aplicações de vacinas.
- Um **pet** pode possuir vários tratamentos.
- Um **médico veterinário** pode ser responsável por vários tratamentos.
- Um **tratamento** pode estar relacionado a uma consulta.

As chaves estrangeiras são utilizadas para estabelecer os relacionamentos entre as tabelas e garantir a integridade referencial dos dados.



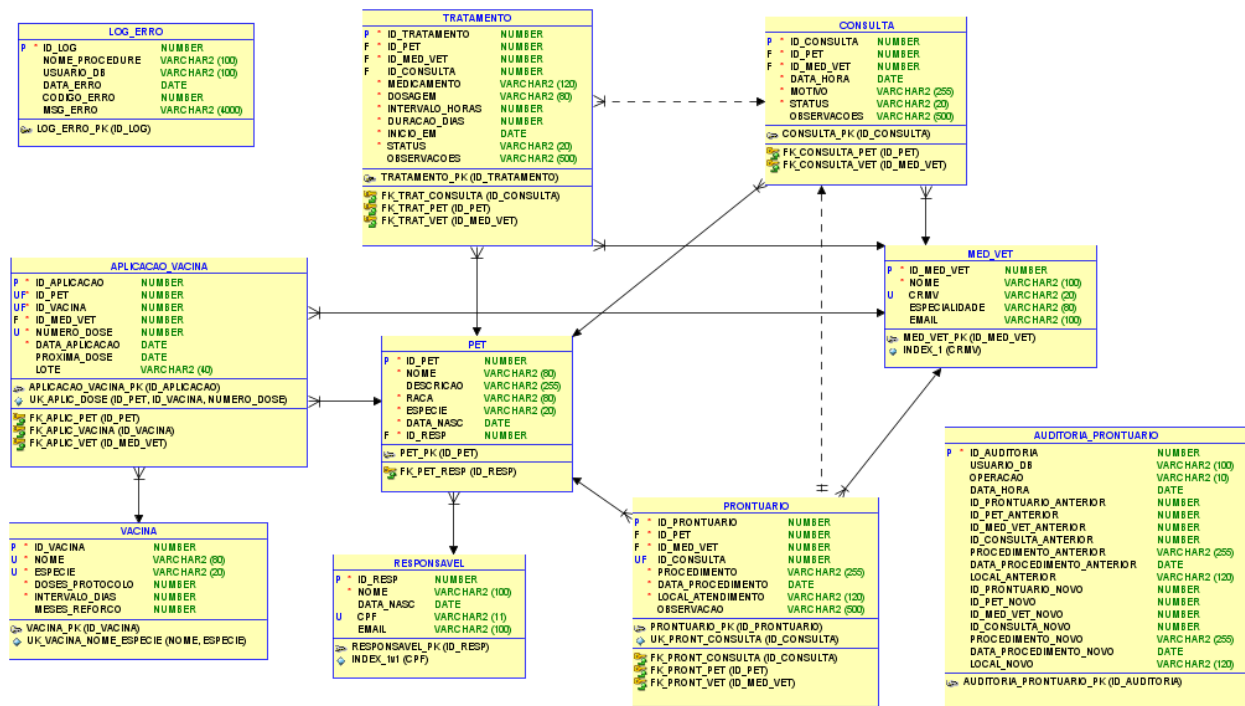
3. Modelo Físico (MER)

O Modelo Físico apresenta a implementação do modelo lógico no banco de dados Oracle. Nesta etapa são definidas as tabelas, atributos, tipos de dados, chaves primárias, chaves estrangeiras e demais restrições de integridade utilizadas na estrutura.

O banco de dados é composto por tabelas destinadas ao armazenamento dos dados operacionais do sistema e por tabelas auxiliares utilizadas para o tratamento de erros e auditoria.

Entre as principais restrições implementadas estão as chaves primárias para identificação dos registros, chaves estrangeiras para estabelecer os relacionamentos entre as entidades e restrições de unicidade para atributos como CPF, CRMV e a combinação de nome e espécie das vacinas.

O modelo físico foi desenvolvido com base no modelo lógico e implementado utilizando Oracle SQL.



4. Implementação SQL

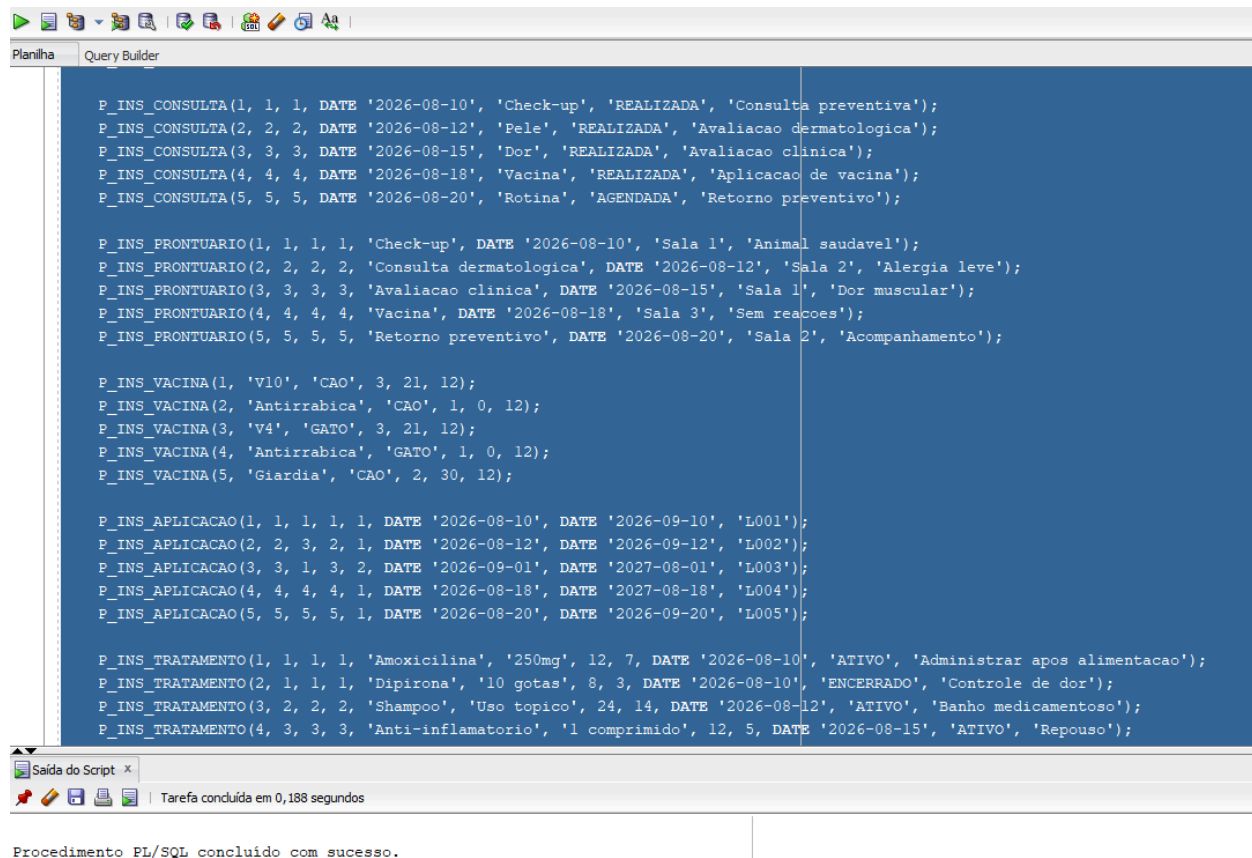
4.1. Estrutura e carga de dados

A estrutura do banco de dados foi implementada em Oracle SQL, contemplando tabelas, chaves primárias, chaves estrangeiras, restrições de unicidade e relacionamentos entre os dados.

Foram criadas as tabelas **RESPONSAVEL**, **MED_VET**, **PET**, **CONSULTA**, **PRONTUARIO**, **VACINA**, **APLICACAO_VACINA**, **TRATAMENTO**, **LOG_ERRO** e **AUDITORIA_PRONTUARIO**.

A carga inicial foi realizada por meio de procedures específicas de inserção, permitindo centralizar o processo e realizar o tratamento de exceções. Foram inseridos dados relacionados aos responsáveis, médicos veterinários, pets, consultas, prontuários, vacinas, aplicações de vacinas e tratamentos.

Cada tabela principal utilizada no projeto possui registros suficientes para permitir a execução das consultas e funcionalidades desenvolvidas.



```
P_INS_CONSULTA(1, 1, 1, DATE '2026-08-10', 'Check-up', 'REALIZADA', 'Consulta preventiva');
P_INS_CONSULTA(2, 2, 2, DATE '2026-08-12', 'Pele', 'REALIZADA', 'Avaliacao dermatologica');
P_INS_CONSULTA(3, 3, 3, DATE '2026-08-15', 'Dor', 'REALIZADA', 'Avaliacao clinica');
P_INS_CONSULTA(4, 4, 4, DATE '2026-08-18', 'Vacina', 'REALIZADA', 'Aplicacao de vacina');
P_INS_CONSULTA(5, 5, 5, DATE '2026-08-20', 'Rotina', 'AGENDADA', 'Retorno preventivo');

P_INS_PRONTUARIO(1, 1, 1, 1, 'Check-up', DATE '2026-08-10', 'Sala 1', 'Animal saudavel');
P_INS_PRONTUARIO(2, 2, 2, 2, 'Consulta dermatologica', DATE '2026-08-12', 'Sala 2', 'Alergia leve');
P_INS_PRONTUARIO(3, 3, 3, 3, 'Avaliacao clinica', DATE '2026-08-15', 'Sala 1', 'Dor muscular');
P_INS_PRONTUARIO(4, 4, 4, 4, 'Vacina', DATE '2026-08-18', 'Sala 3', 'Sem reacoes');
P_INS_PRONTUARIO(5, 5, 5, 5, 'Retorno preventivo', DATE '2026-08-20', 'Sala 2', 'Acompanhamento');

P_INS_VACINA(1, 'V10', 'CAO', 3, 21, 12);
P_INS_VACINA(2, 'Antirrabica', 'CAO', 1, 0, 12);
P_INS_VACINA(3, 'V4', 'GATO', 3, 21, 12);
P_INS_VACINA(4, 'Antirrabica', 'GATO', 1, 0, 12);
P_INS_VACINA(5, 'Giardia', 'CAO', 2, 30, 12);

P_INS_APLICACAO(1, 1, 1, 1, 1, DATE '2026-08-10', DATE '2026-09-10', 'L001');
P_INS_APLICACAO(2, 2, 3, 2, 1, DATE '2026-08-12', DATE '2026-09-12', 'L002');
P_INS_APLICACAO(3, 3, 1, 3, 2, DATE '2026-09-01', DATE '2027-08-01', 'L003');
P_INS_APLICACAO(4, 4, 4, 4, 1, DATE '2026-08-18', DATE '2027-08-18', 'L004');
P_INS_APLICACAO(5, 5, 5, 5, 1, DATE '2026-08-20', DATE '2026-09-20', 'L005');

P_INS_TRATAMENTO(1, 1, 1, 1, 'Amoxicilina', '250mg', 12, 7, DATE '2026-08-10', 'ATIVO', 'Administrar apos alimentacao');
P_INS_TRATAMENTO(2, 1, 1, 1, 'Dipirona', '10 gotas', 8, 3, DATE '2026-08-10', 'ENCERRADO', 'Controle de dor');
P_INS_TRATAMENTO(3, 2, 2, 2, 'Shampoo', 'Uso topico', 24, 14, DATE '2026-08-12', 'ATIVO', 'Banho medicamentoso');
P_INS_TRATAMENTO(4, 3, 3, 3, 'Anti-inflamatorio', '1 comprimido', 12, 5, DATE '2026-08-15', 'ATIVO', 'Repouso');
```

Procedimento PL/SQL concluído com sucesso.

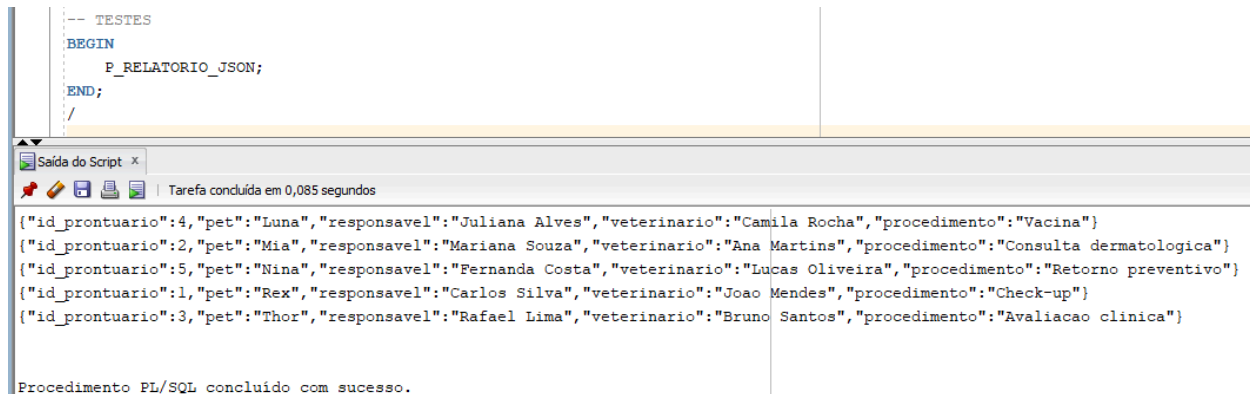
4.2. Procedures

4.2.1. P_RELATORIO_JSON

A procedure **P_RELATORIO_JSON** gera um relatório dos prontuários cadastrados no banco de dados e realiza a integração entre diferentes tabelas relacionais.

Para obter os dados necessários, a procedure utiliza **JOIN** entre as tabelas **PRONTUARIO**, **PET**, **RESPONSAVEL** e **MED_VET**. Os registros são percorridos por meio de um cursor implícito e enviados para a função **F_JSON_ATENDIMENTO**, responsável pela conversão manual dos dados relacionais para uma string no formato JSON.

Antes da geração do relatório, é realizada uma verificação da quantidade de registros existentes na tabela **PRONTUARIO**. O tratamento de exceções contempla situações como ausência de registros, erro de valor e outras exceções não previstas.



```
-- TESTES
BEGIN
    P_RELATORIO_JSON;
END;
```

Tarefa concluída em 0,085 segundos

```
{
  "id_prontuario":4,"pet":"Luna","responsavel":"Juliana Alves","veterinario":"Camila Rocha","procedimento":"Vacina"
},
{
  "id_prontuario":2,"pet":"Mia","responsavel":"Mariana Souza","veterinario":"Ana Martins","procedimento":"Consulta dermatologica"
},
{
  "id_prontuario":5,"pet":"Nina","responsavel":"Fernanda Costa","veterinario":"Lucas Oliveira","procedimento":"Retorno preventivo"
},
{
  "id_prontuario":1,"pet":"Rex","responsavel":"Carlos Silva","veterinario":"Joao Mendes","procedimento":"Check-up"
},
{
  "id_prontuario":3,"pet":"Thor","responsavel":"Rafael Lima","veterinario":"Bruno Santos","procedimento":"Avaliacao clinica"
}
```

Procedimento PL/SQL concluído com sucesso.

4.2.2. P_RELATORIO_TRATAMENTOS

A procedure **P_RELATORIO_TRATAMENTOS** realiza um relatório hierárquico dos tratamentos registrados no banco de dados.

A consulta utiliza a tabela **TRATAMENTO**, considerando as categorias **ID_PET** e **STATUS**, além do atributo numérico **DURACAO_DIAS**. A procedure utiliza um cursor explícito para percorrer os registros ordenados por pet e status.

Durante o processamento, são calculados manualmente os subtotais de duração para cada status, o subtotal de cada pet e o total geral. A lógica é implementada diretamente na procedure, sem utilização de **ROLLUP**, **CUBE** ou **GROUPING SETS**.

O processamento também possui tratamento de exceções para ausência de dados, erros de valor e outras exceções.

```
BEGIN
    P_RELATORIO_TRATAMENTOS;
END;
/
```

Saída do Script x
Tarefa concluída em 0,081 segundos

1	ATIVO	7
1	ENCERRADO	3
1	T	10
2	ATIVO	14
2	ENCERRADO	4
2	T	18
3	ATIVO	5
3	ENCERRADO	6
3	T	11
4	ATIVO	10
4	ENCERRADO	8
4	T	18
5	ATIVO	5
5	T	5
Total		62

Procedimento PL/SQL concluído com sucesso.

4.3. Functions

Foram desenvolvidas duas funções para complementar as funcionalidades do banco de dados. A primeira realiza a conversão manual de dados relacionais para JSON e a segunda executa um cálculo relacionado à duração de tratamentos.

4.3.1. F_JSON_ATENDIMENTO

A função **F_JSON_ATENDIMENTO** recebe os dados de um atendimento e retorna uma string estruturada manualmente no formato JSON.

A função consulta o prontuário pelo seu identificador e valida os dados recebidos. A construção do JSON é realizada por concatenação de strings, sem utilização das funções nativas de geração ou consulta de JSON do Oracle.

O tratamento de exceções contempla prontuário inexistente, dados inválidos e outras exceções.

```
SELECT F_JSON_ATENDIMENTO(1, 'Pet Teste', 'Responsavel Teste', 'Veterinario Teste', 'Consulta veterinaria') FROM DUAL;
```

Resultado da Consulta x

Todas as Linhas Extraídas: 1 em 0,024 segundos

```
F_JSON_ATENDIMENTO(1,PETTESTE,RESPONSAVELTESTE,VETERINARIOTESTE,CONSULTAVETERINARIA)
1 [{"id_prontuario":1,"pet":"Pet Teste","responsavel":"Responsavel Teste","veterinario":"Veterinario Teste","procedimento":"Consulta veterinaria"}]
```



```
SELECT F_JSON_ATENDIMENTO(999, 'Pet', 'Responsavel', 'Veterinario', 'Consulta') FROM DUAL;
```

Resultado da Consulta x

Todas as Linhas Extraídas: 1 em 0,074 segundos

```
F_JSON_ATENDIMENTO(999,PET,RESPONSAVEL,VETERINARIO,CONSULTA)
1 [{"erro":"prontuario nao encontrado"}]
```

4.3.2. F_FIM_TRATAMENTO

A função **F_FIM_TRATAMENTO** calcula a data prevista para o término de um tratamento a partir da data de início e da duração em dias.

A função valida a data inicial e a duração informada. Valores nulos ou negativos são tratados como inválidos, assim como uma duração igual a zero.

O resultado é obtido pela soma da quantidade de dias à data de início. O tratamento de exceções retorna valor nulo para situações inválidas ou não previstas.

```
SELECT F_FIM_TRATAMENTO (DATE '2026-09-13', 0) FROM DUAL;
```

Resultado da Consulta x

Todas as Linhas Extraídas: 1 em 0,021 segundos

```
F_FIM_TRATAMENTO (DATE'2026-09-13',0)
1 (null)
```



```
SELECT F_FIM_TRATAMENTO (DATE '2026-09-13', -2) FROM DUAL;
```

Resultado da Consulta x

Todas as Linhas Extraídas: 1 em 0,02 segundos

```
F_FIM_TRATAMENTO (DATE'2026-09-13',-2)
1 (null)
```

4.4. Trigger

Foi implementado o trigger **TRG_AUDITORIA_PRONTUARIO**, associado à tabela **PRONTUARIO**.

O trigger é executado após operações de **INSERT**, **UPDATE** ou **DELETE** e registra automaticamente as informações da operação na tabela **AUDITORIA_PRONTUARIO**.

Para operações de inserção, são armazenados os novos valores do registro. Para alterações, são registrados os valores anteriores e os novos valores. Para exclusões, são armazenados os valores existentes antes da remoção.

Além dos dados alterados, o registro de auditoria armazena o usuário responsável pela operação, o tipo da operação e a data e horário em que ela ocorreu.

Para demonstrar o funcionamento do trigger, foi realizado um conjunto de testes contendo uma inserção, uma atualização e uma exclusão de um registro de prontuário.

```
BEGIN
  P_INS_PRONTUARIO(
    6, 1, 1, NULL, 'Teste auditoria',
    DATE '2026-08-21', 'Sala 1', 'Insercao para auditoria'
  );

  UPDATE PRONTUARIO
  SET OBSERVACAO = 'Atualizacao para auditoria'
  WHERE ID_PRONTUARIO = 6;

  DELETE FROM PRONTUARIO
  WHERE ID_PRONTUARIO = 6;
END;
```

Procedimento PL/SQL concluído com sucesso.

ID_AUDITORIA	USUARIO_DB	OPERACAO	DATA_HORA	ID_PRONTUARIO_ANTERIOR	ID_PET_ANTERIOR	ID_MED_VET_ANTERIOR	ID_CONSULTA_ANTERIOR	PROCEDIMENTO_ANTERIOR	DATA_PROCEDIMENTO_ANTERIOR	LOCAL_ANTERIOR	ID_PRONTUARIO_NOVO	ID_PET_NOVO	ID_MED_VET_NOVO	ID_CONSULTA_NOVO
1	RM563112	INSERT	13/09/26	(null)	(null)	(null)	(null)	(null)	(null)	(null)	6	1	1	(null)
2	RM563112	UPDATE	13/09/26	6	1	1	(null)	Teste auditoria	21/08/26	Sala 1	6	1	1	(null)
3	RM563112	DELETE	13/09/26	6	1	1	(null)	Teste auditoria	21/08/26	Sala 1	(null)	(null)	(null)	(null)

4.5. Consultas Complementares

Além das procedures, functions e trigger, foram desenvolvidas consultas complementares para demonstrar diferentes recursos de consulta e análise de dados relacionais.

Foi utilizada a função analítica **LAG** para obter o registro anterior e **LEAD** para obter o registro posterior, permitindo comparar os pets em relação à ordenação utilizada na consulta.

Também foram utilizadas consultas com **JOIN** e **GROUP BY** para realizar análises sobre os dados cadastrados. Entre elas estão a quantidade de atendimentos realizados por médico veterinário e a quantidade de pets associados a cada responsável.

Essas consultas demonstram a utilização de recursos de manipulação e análise dos dados relacionais presentes no banco.

```
SELECT
    ID_PET,
    NVL(LAG(NOME) OVER(ORDER BY ID_PET), 'VAZIO') ANTERIOR,
    NOME ATUAL,
    NVL(LEAD(NOME) OVER(ORDER BY ID_PET), 'VAZIO') PROXIMO
FROM PET;
```

ID_PET	ANTERIOR	ATUAL	PROXIMO
1	VAZIO	Rex	Mia
2	Rex	Mia	Thor
3	Mia	Thor	Luna
4	Thor	Luna	Nina
5	Luna	Nina	VAZIO

```

BEGIN
  FOR R IN (
    SELECT V.NOME, COUNT(PR.ID_PRONTUARIO) TOTAL_ATENDIMENTOS
    FROM MED_VET V
    JOIN PRONTUARIO PR ON V.ID_MED_VET = PR.ID_MED_VET
    GROUP BY V.NOME
    ORDER BY TOTAL_ATENDIMENTOS DESC
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(R.NOME || ' | ' || R.TOTAL_ATENDIMENTOS);
  END LOOP;
END;
/

```

 Saída do Script x
  Resultado da Consulta x





 | Tarefa concluída em 0,066 segundos

```

Camila Rocha | 1
Bruno Santos | 1
Ana Martins | 1
Joao Mendes | 1
Lucas Oliveira | 1

```

Procedimento PL/SQL concluído com sucesso.

```

BEGIN
  FOR R IN (
    SELECT R.NOME, COUNT(P.ID_PET) TOTAL_PETS
    FROM RESPONSAVEL R
    JOIN PET P ON R.ID_RESP = P.ID_RESP
    GROUP BY R.NOME
    ORDER BY TOTAL_PETS DESC
  ) LOOP
    DBMS_OUTPUT.PUT_LINE(R.NOME || ' | ' || R.TOTAL_PETS);
  END LOOP;
END;
/

```

 Saída do Script x
  Resultado da Consulta x





 | Tarefa concluída em 0,112 segundos

```

Carlos Silva | 1
Mariana Souza | 1
Juliana Alves | 1
Rafael Lima | 1
Fernanda Costa | 1

```

Procedimento PL/SQL concluído com sucesso.

5. Observações

O banco de dados foi implementado em Oracle SQL, contemplando a criação das estruturas relacionais, carga dos dados, procedures, functions e trigger de auditoria.

A implementação inclui procedures para geração de relatórios, utilização de JOINS e cursores, conversão manual de dados relacionais para JSON, cálculo de subtotais e total geral, além de uma função para cálculo da data de término de tratamentos.

Também foi implementado um mecanismo de tratamento de exceções e registro de erros na tabela LOG_ERRO. O trigger de auditoria permite registrar operações de inserção, alteração e exclusão realizadas na tabela PRONTUARIO, armazenando os valores anteriores e novos quando aplicável.

As consultas complementares demonstram a utilização de funções analíticas, como LAG e LEAD, além de consultas com JOIN e GROUP BY.

O arquivo SQL disponibilizado contém os comandos necessários para criação das tabelas, inserção dos dados, criação das procedures, functions, trigger e execução dos testes apresentados neste documento.